COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | 08/04/2026 | **DATE SUBMITTED:** | 08/04/2026 |
| **NAME:** | Jarel Dave R. Egar | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. ___ [ACTIVITY NAME]**

## 1. PROCEDURE OUTPUT

ILO A

Screenshots

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);


int main(){

  std::queue<char> q;
  q.push('c');
  q.push('p');
  q.push('e');
  q.push('0');
  q.push('1');
  q.push('0');


  //DISPLAY
  display(q);

//EMPTY

std::cout << "The queue is empty: " << q.empty();
std::cout<< "\n";

//SIZE

std::cout << "Queue size: " << q.size();
std::cout<< "\n";

//FRONT

std::cout << "The front is: " << q.front();
std::cout<< "\n";

//BACK

std::cout << "The back is: " << q.back();
std::cout<< "\n";

//POP

q.pop();

//DISPLAY

display(q);
```

| | |
|---|---|
| |  |
| Output |  |

```
  display(q);

  //PUSH ANOTHER ELEMENT

  q.push('J');
  q.push('A');

  //DISPLAY

  display(q);

  }

void display(std::queue<char> r){
  //create a copy of the queue
  std::queue<char> c = r;

  while(!c.empty()){
    std::cout<< " " << c.front();
    c.pop();

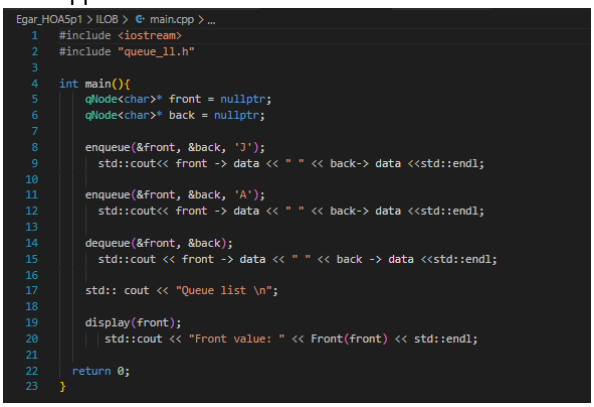
  }
  std::cout << "\n";

}
```

```
In file included from main.cpp:2:
queue_ll.h: In function 'T Front(qNode<T>*) [with T = char]':
queue_ll.h:84:1: warning: control reaches end of non-void function [-Wreturn-type]
   84 | }
      | ^
J A
J A
J
PS C:\Users\TIPQC\Documents\Egar_CPE2151\Egar_HOA5p1\ILOB>
```

Discussion and code analysis: This activity provided insights for queues and how to use them for arrays and linked lists. During this activity when coding for an array I had an easy time, however for the linked list I had multiple error and had trouble finishing it.

ILOB part 1

| | |
|---|---|
| Screenshot | Main.cpp<br>  <br>QueueLL_h |

```
Egar_HOA5p1 > ILOB > main.cpp > ...
#include <iostream>
#include "queue_ll.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue(&front, &back, 'J');
      std::cout<< front -> data << " " << back-> data <<std::endl;

    enqueue(&front, &back, 'A');
      std::cout<< front -> data << " " << back-> data <<std::endl;

    dequeue(&front, &back);
      std::cout << front -> data << " " << back -> data <<std::endl;

    std:: cout << "Queue list \n";

    display(front);
      std::cout << "Front value: " << Front(front) << std::endl;

  return 0;
}
```

```
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
  public:
  T data;
  qNode* next;
};


template<typename T>
qNode<T>* new_node(T newData){
  qNode<T>* newNode = new qNode<T>;
  newNode -> data = newData;
  newNode -> next = nullptr;
  return newNode;

}

template<typename T>
void enqueue(qNode<T> **frontPtr, qNode<T> ** backPtr, T newData){

  // create new node
    qNode<T> *newNode = new_node<T>(newData);

  //inserting to an empty queue
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr){
    (*frontPtr) = newNode;
    (*backPtr) = newNode;

    }

  //inserting an item into a non empty queue
  (*backPtr)-> next = newNode;
  (*backPtr) = newNode;

}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //create temp node to store the node to be deleted
    qNode<T> *deleteNode = nullptr;
    deleteNode = (*frontPtr);
```

```
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //create temp node to store the node to be deleted
    qNode<T> *deleteNode = nullptr;
    deleteNode = (*frontPtr);

    //checki if the queue is only one node
      if((*frontPtr) -> next == nullptr && (*backPtr) -> next == nullptr)
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;

    // deleting of the node
    (*frontPtr) = deleteNode -> next;
    delete deleteNode;

}

//Display all elements in the list

template <typename T>
void display(qNode<T>* frontPtr){
   if(frontPtr == nullptr){
       std::cout<< "queue is empty \n";
       return;
   }

   qNode<T>* current = frontPtr;
   while (current != nullptr){
    std::cout << current->data << "->";
    current= current->next;
   }
   std::cout<< "NULL \n";

}

// return the front
template <typename T>
T Front(qNode<T>* frontPtr){

   if(frontPtr != nullptr) {
    return frontPtr->data;
   }
}

#endif
```

Output

```
PS D:\VSCode\Projects\Egar_HOASp1\ILO8> cd "d:\VSCode\Projects\Egar_HOASp1\ILO8\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
In file included from main.cpp:2:
queue_ll.h: In function 'T Front(qNode<T>*) [with T = char]':
queue_ll.h:84:1: warning: control reaches end of non-void function [-Wreturn-type]
   84 | }
      | ^
J J
J A
J
PS D:\VSCode\Projects\Egar_HOASp1\ILO8>
```

Discussion and code analysis:

| Screenshot | Main.cpp |
|---|---|

Egar_HOA5p1 > ILOBpart2 > main.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    //create an objecy

//enqueue
    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(2);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(3);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(4);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(5);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(6);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

// dequeue
    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
    return 0;
```

```cpp
// copy constructor
    queueArr<int> q2(q);
    std::cout << "q2_front: " << q2.Front() << ", q2_back: " << q2.Back() << ", q2_size: " << q2.size() << std::endl;

// assignment operator
    queueArr<int> q3(6);
    q3 = q;
    std::cout << "q3_front: " << q3.Front() << ", q3_back: " << q3.Back() << ", q3_size: " << q3.size() << std::endl;

}
```

queueArr.h

Egar_HOA5p1 > ILOBpart2 > queueArr.h > operator=(const queueArr &)

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>
template <typename T>
class queueArr{
    private:
        // create apointer array:
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;


    public:
        //prototypes to use for queueArr:
        queueArr(size_t capacity);
        bool empty();
        bool full();

        size_t size();
        void Clear();

        T Front();
        T Back();

        void Enqueue(T newData);
        void Dequeue();

        ~queueArr();



        //tasks
        //copy constructor
        queueArr(const queueArr& other);

        //assignment operator
        queueArr& operator=(const queueArr& other);
};

```

```cpp
// constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){

    //capacity of the array
    q_capacity = capacity;

    //current size of the array
    q_size = 0;

    //initial front and back of the index
    q_front = 0;
    q_back = -1;

    // newly allocated array
    q_array = new T[q_capacity];
    std::cout << "Queue created with capacity: " << q_capacity << std::endl;
}

//destructor
template <typename T>
queueArr<T>::~queueArr(){

    delete [] q_array;
    std::cout << "Queue destroyed." << std::endl;
}


//empty
template <typename T>
bool queueArr<T>::empty(){
    return q_size == 0;
}

//full
template <typename T>
bool queueArr<T>::full(){
    return q_size == q_capacity;
}
```

```cpp
//clear
template <typename T>
void queueArr<T>::Clear(){
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

//return the element in the front of the queue
template <typename T>
T queueArr<T>::Front(){
    //check and display if the queue is empty
    if (empty()){
        std::cout << "Queue is empty. \n" << std::endl;
        return T{}; // return default value of T
    }
    //return the front element
    return q_array[q_front];
}

// back
template <typename T>
T queueArr<T>::Back(){
    //check and display if the queue is empty
    if (empty()){
        std::cout << "Queue is empty. \n" << std::endl;
        return T{}; // return default value of T
    }
    // if not then return the back element
    return q_array[q_back];
}

template <typename T>
size_t queueArr<T>::size(){
    return q_size;
}
```

```cpp
template <typename T>
void queueArr<T>::Enqueue(T newData){
    // check if the queue is full
    if (full()){
        std::cout << "Queue is full." << std::endl;
        return;
    }

    // compute for the q_back to move it circularly
    q_back = (q_back + 1) % q_capacity;

    //assign the new value to the back of the queue array
    q_array[q_back] = newData;
    //increment the q_size
    q_size++;
}

//dequeue
template <typename T>
void queueArr<T>::Dequeue(){
    // check if the queue is empty
    if (empty()){
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    // compute for q_front to move it circularly
    q_front = (q_front + 1) % q_capacity;

    //decrement the size q_size
    q_size--;
}

```

Output

```
main.cpp
Egar_HOASp1 > ILOBpart2 > main.cpp > main()
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\VSCode\Projects> cd "d:\VSCode\Projects\Egar_HOASp1\ILOBpart2\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Queue created with capacity: 6
q_front: 1, q_back: 1, q_size: 1
q_front: 1, q_back: 2, q_size: 2
q_front: 1, q_back: 3, q_size: 3
q_front: 1, q_back: 4, q_size: 4
q_front: 1, q_back: 5, q_size: 5
q_front: 1, q_back: 6, q_size: 6
2, q_back: 6, q_size: 5
3, q_back: 6, q_size: 4
4, q_back: 6, q_size: 3
5, q_back: 6, q_size: 2
6, q_back: 6, q_size: 1
Queue is empty.

0, q_back: Queue is empty.

0, q_size: 0
Queue is empty.
Queue is empty.

0, q_back: Queue is empty.

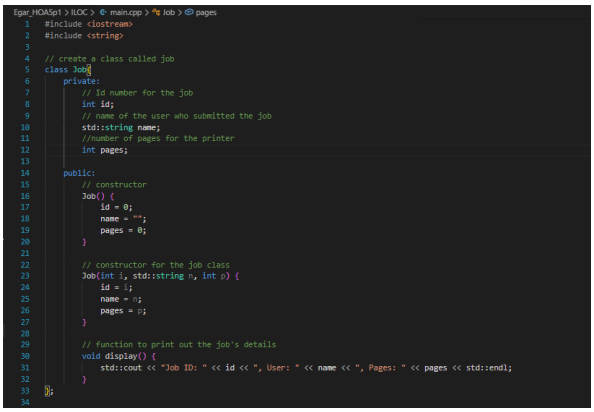
0, q_size: 0
Queue destroyed.
PS D:\VSCode\Projects\Egar_HOASp1\ILOBpart2>
```

**Conclusion:** During this activity, I learned how to use a queue and understand how the enqueue and dequeue functions work. I also learned how to check the front, back, and size of the queue while adding and removing elements.

I learned how to use an array to store the elements and how the queue follows the First-In, First-Out (FIFO) method. I also learned how the copy constructor and assignment operator can be used to copy a queue.

ILO C / SUPPLEMENTARY

Screenshot



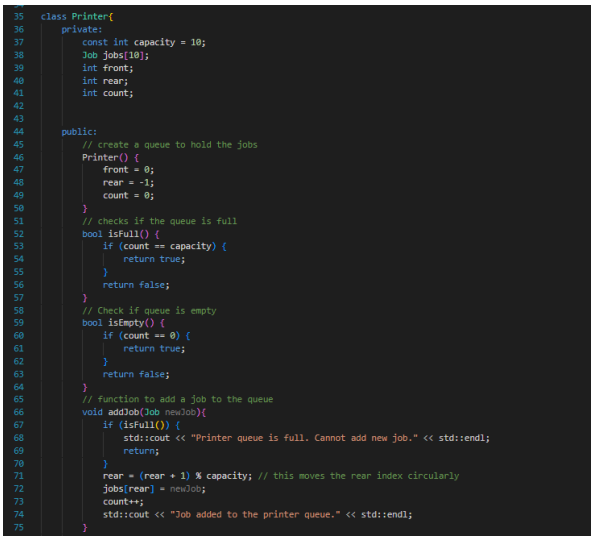
```
Egar_HOASp1 > ILOC > main.cpp > Job > pages
#include <iostream>
#include <string>

// create a class called job
class Job{
    private:
        // Id number for the job
        int id;
        // name of the user who submitted the job
        std::string name;
        //number of pages for the printer
        int pages;

    public:
        // constructor
        Job() {
            id = 0;
            name = "";
            pages = 0;
        }

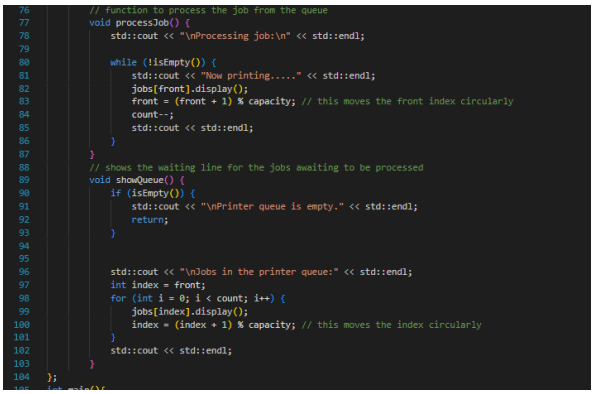
        // constructor for the job class
        Job(int i, std::string n, int p) {
            id = i;
            name = n;
            pages = p;
        }

        // function to print out the job's details
        void display() {
            std::cout << "Job ID: " << id << ", User: " << name << ", Pages: " << pages << std::endl;
        }
};
```



```
class Printer{
    private:
        const int capacity = 10;
        Job jobs[10];
        int front;
        int rear;
        int count;


    public:
        // create a queue to hold the jobs
        Printer() {
            front = 0;
            rear = -1;
            count = 0;
        }
        // checks if the queue is full
        bool isFull() {
            if (count == capacity) {
                return true;
            }
            return false;
        }
        // Check if queue is empty
        bool isEmpty() {
            if (count == 0) {
                return true;
            }
            return false;
        }
        // function to add a job to the queue
        void addJob(Job newJob){
            if (isFull()) {
                std::cout << "Printer queue is full. Cannot add new job." << std::endl;
                return;
            }
            rear = (rear + 1) % capacity; // this moves the rear index circularly
            jobs[rear] = newJob;
            count++;
            std::cout << "Job added to the printer queue." << std::endl;
        }
```



```
        // function to process the job from the queue
        void processJob() {
            std::cout << "\nProcessing job:\n" << std::endl;

            while (!isEmpty()) {
                std::cout << "Now printing....." << std::endl;
                jobs[front].display();
                front = (front + 1) % capacity; // this moves the front index circularly
                count--;
                std::cout << std::endl;
            }
        }
        // shows the waiting line for the jobs awaiting to be processed
        void showQueue() {
            if (isEmpty()) {
                std::cout << "\nPrinter queue is empty." << std::endl;
                return;
            }


            std::cout << "\nJobs in the printer queue:" << std::endl;
            int index = front;
            for (int i = 0; i < count; i++) {
                jobs[index].display();
                index = (index + 1) % capacity; // this moves the index circularly
            }
            std::cout << std::endl;
        }
};
```

```cpp
int main(){
    //creating of objects for the printer and jobs
    Printer printer;


    int numberOfJobs;
    std::cout << "Enter the number of jobs to add to the printer queue: ";
    std::cin >> numberOfJobs;

    std::cout << std::endl;


    for (int i = 0; i < numberOfJobs; i++) {
        int numPages;
        std::string name;


        std::cout << "Enter User Name: " << (i + 1) << ": ";
        std::cin >> name;
        std::cout << "Enter Number of Pages: " << (i + 1) << ": ";
        std::cin >> numPages;


        Job newJob(i + 1, name, numPages);
        printer.addJob(newJob);
        std::cout << std::endl;
    }


    // Show the jobs in the queue
    printer.showQueue();

    // Process the jobs in the queue
    printer.processJob();

    // Show the jobs in the queue after processing
    printer.showQueue();


    return 0;
}
```

Output

```
Enter the number of jobs to add to the printer queue: 3

Enter User Name: 1: Ja
Enter Number of Pages: 1: 2
Job added to the printer queue.

Enter User Name: 2: rel
Enter Number of Pages: 2: 3
Job added to the printer queue.

Enter User Name: 3: Brei
Enter Number of Pages: 3: 4
Job added to the printer queue.


Jobs in the printer queue:
Job ID: 1, User: Ja, Pages: 2
Job ID: 2, User: rel, Pages: 3
Job ID: 3, User: Brei, Pages: 4


Processing job:

Now printing.....
Job ID: 1, User: Ja, Pages: 2

Now printing.....
Job ID: 2, User: rel, Pages: 3

Now printing.....
Job ID: 3, User: Brei, Pages: 4


Printer queue is empty.
PS D:\VSCode\Projects\Egar_HOASp1\ILOC>
```

**Discussion/conclusion:** I chose an array over a linked list because arrays are much easier to work with. With an array, I can simply declare the exact capacity I need from the start. On the other hand, while a linked list offers flexible sizing, manually creating and managing nodes can quickly become confusing.

However, I realized during this activity that I still need to improve in a few areas. I noticed that I tend to forget specific syntax, proper class declarations, and the required parameters for methods.

Overall, this activity helped me better understand how to use arrays effectively and highlighted their simplicity compared to linked lists.

COMPUTER ENGINEERING

QUEZON CITY

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Claude (ILOC)**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Understanding the how to code, explaining the error**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I used claude to explain how I should start the activity and asked it to explain the error but don't show the fixed version for the code**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.